



UNIVERSIDADE FEDERAL DO ESPÍRITO SANTO
CENTRO DE CIÊNCIAS EXATAS, NATURAIS E DA SAÚDE
DEPARTAMENTO DE COMPUTAÇÃO

TÓPICOS ESPECIAIS EM PROGRAMAÇÃO I
Prof. Giuliano Prado de Moraes Giglio

EXERCÍCIOS FIXAÇÃO DE OO com JS

1. Através de FUNÇÕES CONSTRUTORAS e, de acordo com o que foi passado de conceitos da OO até então, construa os seguintes objetos:

Livro: todo livro possui os dados: **numero de ordem, titulo, editora, valor de locação, tipo, quantidade**. Os métodos a serem implementados são:

calculaValorTotal (): calcula o valor total de cada livro. Esse cálculo é o valor da locação multiplicado pela quantidade de livros, retornando esta quantidade.

método exhibir(): exibe as informações do livro.

darBaixa(valor): entra com um valor que corresponde a quantidade que se dará baixa, ou seja, diminuirá da quantidade de livros. Deve-se verificar se o valor fornecido por parâmetro é menor do que a quantidade de livros do atributo do objeto. Caso seja, retirar da quantidade o valor passado e retornar **true**. Caso contrário, só retorna **false**.

Após a criação da função construtora acima, pede-se:

- I. Criar 3 objetos livros.
- II. Calcule o valor total de cada livro, mostrando esse resultado juntamente com o seu numero de ordem e nome.
- III. Peça ao usuário para digitar um valor que deseja dar baixa em um livro e aplique em cada livro criado. Mostre as informações de todos aqueles livros que foi possível retirar o valor fornecido do seu estoque.

2. Usando FUNÇÃO FACTORY, crie o seguinte objeto descrito abaixo:

Funcionário: todo funcionário da empresa possui os atributos **CPF, nome, salário base, cargo e numero de filhos**. Os métodos a serem implementados são:

Passagem de parâmetros: a função não terá passagem de qualquer parâmetro para os atributos. Internamente, o atributo **CPF** será inicializado com “000.000.000-11” e o **nome** com o valor “INDEFINIDO”

método exibirFuncionario(): mostrar na tela todos os atributos de um objeto funcionário.

método exibirIdentificacao(): mostrar na tela somente os atributos **nome** e **CPF** do objeto funcionário.

calcularImposto (): o valor do imposto é calculado a partir do salário base do funcionário. Caso este salário seja inferior a \$1500,00 (inclusive), não há imposto. Caso estiver entre \$1500,00 e \$2750,00 (inclusive), paga-se 12% sobre o valor do salário. Caso o salário base seja superior a \$2750,00 o imposto é de 23,5% do valor do salário. O método deverá retornar o valor do imposto calculado.

calcularSalarioBruto(): o valor do salário bruto é o salário base acrescido de \$120,00 para cada filho do funcionário, menos 11% de INSS sobre o valor do salário base. O método deverá retornar este valor.

Teste a criação de objetos, da seguinte forma:

- I. Crie 2 objetos funcionário. Complete as informações de cada objeto criado, atributo por atributo, pedindo as informações ao usuário.
- II. Calcule os impostos de cada funcionário e imprime na tela o nome e CPF do funcionário que possui o maior valor desse imposto a pagar.
- III. Calcule o salário bruto de cada objeto, mostrando-o na tela junto com todas as informações de cada objeto.

3. A empresa **DataHouse** deseja que você faça um programa que controle a folha de pagamento de seus ANALISTAS. Para isso, ela deseja fazer uma simulação com um objeto que representa um Analista para avaliar esse cálculo, para depois replicar aos demais. Logo, você deverá implementar um código JS da seguinte forma:

Usando NOTAÇÃO LITERAL, crie um objeto **Analista**, que terá os seguintes atributos: código do funcionário, nome, cargo, departamento, número de dependentes, ano de admissão, idade. Na criação, todo objeto Analista deverá estar lotado no departamento de “Tecnologia”

Esta classe possui as seguintes operações (métodos):

- i. **Exibir informações do Analista:** mostra na tela os dados cadastrais do funcionário, com exceção do número de dependentes;
- ii. **Calcular gratificação por cargo:** o analista pode ocupar somente três cargos: Técnico, Analista, Projetista. Cada cargo possui um valor que será acrescido ao salário bruto do funcionário, correspondendo a sua gratificação, da seguinte forma:

Técnico = + \$500,00 Analista = + \$800,00 Projetista = + 1.100,00

Ao final, deve-se retornar o novo salário bruto.

- iii. **Calcular o salário líquido:** o salário líquido do Analista é calculado descontando do salário bruto o 12% do INSS e somando o salário família, que é o número de dependentes multiplicado por uma porcentagem por dependente, esta fornecida por parâmetro.

Com esse objeto criado, use-o para:

- a. Alterar seu salário. Peça ao usuário um novo salário e o altere;
- b. Calcular a gratificação do objeto e mostre esse valor. Como o objeto está lotado no departamento de “Tecnologia”, modifique para o objeto esse departamento para os outros dois previstos, calcule a gratificação para cada cargo, mostrando seu valor.
- c. Calcular o salário líquido do Analista, mostrando seu valor, pedindo ao usuário a sua porcentagem por dependente.
- d. Mostrar todas as informações do objeto Analista;

4. Uma empresa que oferece cursos de especialização para uma universidade necessita de uma aplicação para controlar os seus alunos e matrícula destes. Como parte da aplicação, você deverá trabalhar com os seguintes requisitos para os objetos pedidos:

Obs.: partes do exercício consistem em utilizar funções de manipulação de Strings. Para isso, você pode acessar a documentação do JS (links indicados nos slides) e material complementar fornecido sobre o assunto.

Objeto **Aluno**: todo aluno da instituição possui os dados **numero de identificação individual, nome, valor mensalidade, data de ingresso, semestre letivo, curso, email, senha**.

Métodos:

Reajustar mensalidade: aplica um aumento de sua mensalidade, de acordo com o percentual passado pelo parâmetro, atualizando assim o atributo correspondente. Passa-se por parâmetro o percentual real, ex: 30 que equivale a 30%; 50 que equivale a 50 % e assim por diante.

Saída de suas informações: monte uma saída mostrando os dados do objeto, de maneira formatada e apresentável ao usuário.

Validar aluno: esse método faz-se a validação do aluno, passando por parâmetro um valor correspondente ao email e outra a senha. No método você deverá verificar se os valores dos parâmetros passados correspondem ao email e senha do aluno cadastrado, informando ao usuário se é ou não.

Métodos GET/SET:

- Email deve ser sempre armazenado em maiúsculo e recuperado em minúsculo. Além disso, precisa-se verificar se é um email, tendo @ e pelo menos ter **.com** ou **.br** ou **.edu**
- A senha deverá ter no mínimo 8 caracteres e no máximo 12, ter uma letra maiúscula e um dígito.
- Numero de matricula possui 7 caracteres, sendo os 3 primeiros obrigatoriamente compostos de dígitos (*pesquise como verificar se um caracter é um dígito em JS*).

USANDO HERANÇA DE OBJETOS, crie um novo objeto:

Objeto **AlunoPosGraduacao**: um Aluno Pós Graduação nada mais é do que um Aluno da instituição (ou seja, herda de Aluno acima suas características) que possui alguns atributos específicos, tais como:

Possui os atributos **tempo de graduação** (em anos) e **titulação**, com seus métodos de acesso (get/set);

Reajuste de mensalidade diferenciado: o reajuste de mensalidade do aluno de graduação é feito da mesma forma que todos os outros alunos, porém, antes de efetua-lo, deve-se aplicar um desconto de 3% (em cima de sua mensalidade) por cada ano de graduação.

Teremos um segundo objeto, que representará uma **Matrícula**. Este terá uma característica diferenciada. Possui um atributo que representará um Aluno, ou seja, teremos um objeto dentro de outro objeto, que é o conceito da OO chamado AGREGAÇÃO.

Objeto **Matrícula**: toda matrícula possui os dados: **numero de identificação, data de matrícula, carga horária total, aluno** (*atributo do tipo Aluno, ou seja, um objeto da classe Aluno acima*), **quantidade de matrículas feitas**. Os métodos a serem implementados são:

Na definição e criação desse objeto: inicializa a quantidade de matrículas feitas com zero. O numero de identificação, a carga horária total e o módulo serão digitados pelo usuário, quando da sua criação. Os demais dados não serão inicializados.

Método de Saída de dados: monta a saída das informações de uma matrícula, sendo que deverá ter os dados do aluno, juntamente com todos os dados da matrícula.

Cálculo do valor total: retorna o valor total que será pago pelo aluno. Considerando que todo curso da empresa são realizados em 3 dias por semana e 2 horas em cada dia, você deverá verificar em quantos meses o aluno fará o curso, baseado na carga-horária total deste, fazendo o produto pelo valor da mensalidade do aluno.

matricular: passa-se um aluno por parâmetro, coloque a data atual (digitada) na data de matricula e incremente a quantidade de matriculas feitas, passando esse valor para a identificação da matricula.

Após a criação dos objetos acima, sua aplicação deverá fazer:

- I. Criar dois objetos Aluno, um aluno normal e outro de pós graduação.
- II. Escolha um dos alunos acima e matricule-o. para matricular, você deverá criar um objeto Matricula e passe esse Aluno por parâmetro. Coloque a data atual (capturada do sistema) na data de matricula e incremente a quantidade de matriculas feitas, passando esse valor para a identificação da matricula. Mostre os dados dessa matrícula feita.

- III. Após a matrícula realizada, faça o reajuste da mensalidade do aluno matriculado. Para isso, peça o usuário para digitar as credenciais do aluno (email e senha) e faça a sua validação para realizar o reajuste.
- IV. Calcule e mostre o valor total a pagar pelo aluno.